An Intelligent Crowd Density Monitoring and Alert System for Safe Public Gatherings

V. Vigneshwaran
Department Of Computer Science and Engineering
Vels Institute Of Science Technology And Advanced Studies
Chennai,India
vickyrolex91@gmail.com

S. Udhayapradap
Department Of Computer Science and Engineering
Vels Institute Of Science Technology And Advanced Studies
Chennai,India
pradapmgyt@gmail.com

S.M.Sowmiya
Department Of Computer Science and Engineering
Vels Institute Of Science Technology And Advanced Studies
Chennai,India
smsowmiya.se@vistas.ac.in

Abstract—Large public events such as temple festivals and religious processions in Tamil Nadu often go beyond the crowd-control capacity of narrow passages, entrances, and staircases, leading to dangerous stampedes. Currently, crowd management depends mainly on manual observation and a afterwards review of CCTV footage, a method that is unable to reliably identify an increasing danger before it becomes obvious. The project, An Intelligent Crowd Density Monitoring and Alert System for Safe Public Gatherings, suggests a low-cost system based on IoT and AI that continuously monitors high-risk chokepoints by using both a camera and an IR/laser break-beam sensor. The computer vision model based on the camera estimates the real-time crowd density in a given area, while the laser beam sensor separately counts the number of people passing through that point, enabling the system to cross-check its readings even in cases of poor lighting or when the camera is obstructed. Instead of merely responding to the current crowd level, the system monitors the rate at which density is increasing in each area and compares this rate with a safe occupancy limit that is specific to the physical width of that location. If a zone is approaching its danger level, an automatic alert is sent to the event organisers and to the staff in the control room, giving an early warning before the situation becomes critical. The system is constructed using inexpensive and easily obtained hardware, so that it can be put into use by temple trusts and local authorities at various chokepoints rather than depending on a single central monitoring post. The proposed system wants to close the gap between research on crowd monitoring and the real-world situations, in India where public gatherings happen often with limited resources. It does this through the use of sensor fusion setting thresholds which are particular to every zone, as well as predictions based upon trends.

Keywords— IoT, Crowd Density Estimation, Computer Vision, Sensor Fusion, IR/Laser Break-Beam Sensor, Stampede Prediction, Real-Time Alert System, Chokepoint Monitoring, Machine Learning, Smart Surveillance

I. INTRODUCTION

1.1 Background on Crowd Safety in Religious Gatherings

During religious festivals and assembly at the temples, the large congregations of people repeatedly navigating the entrance, line, small spaces, escalators, approach to sanctum sanctorum, and temporary barriers, is a clear pattern of crowd management. A festival isn't one way transport corridor, it's two way, irregular and subject to ritual, announcements, opening of queues and temporary changes in access. Therefore, besides knowing how many people, a monitoring system must also be aware of where are they and how fast is the people changing. Traditional control rooms usually make use of a mix of security personnel, public-address announcements, barricades, and CCTV feeds. Although these methods are still important, the human observers have to combine information from a number of different locations at the same time. A camera view might show a particular area clearly while hiding others, and the operator may have to compare several displays before concluding that a corridor is filling up faster than anticipated. Automated crowd analytics [1], [3], [9], [12] can help by turning the video stream into quantifiable variables such as the number of people, occupancy level, density, and the rate of change.

Studies on crowd counting have shown that it is difficult in densely populated scenes due to occlusion, variations in scale, perspective, and the limited visibility of individual bodies or heads. Density-estimation methods [10], [11] therefore build up a spatial representation rather than depending completely on the detection of individuals. This issue is significant for public safety since the same total number of people can indicate very different spatial arrangements and different degrees of congestion. The system proposed here uses a detection-and-tracking approach because the prototype requires real-time individual counting within a defined area rather than just producing an offline density map.

1.2 Challenges in Temple Crowd Management

Temple crowd environments include practical limitations which are not always reflected in controlled computer vision datasets. Cameras may be positioned at a high angle, a low angle, or from the side, depending on the site. Lighting can vary between day, evening, and indoor situations. Decorative structures, umbrellas, temporary stalls, banners, pillars, and moving objects can partially block people's view. All these can lead to an unreliable count if you were to use vision as your only method. At high densities, the issue of overlapping detections is also a problem. When a lot of people are standing in close proximity, the detector may detect the outline of the boxes that have not been completely detected, or it may not detect a person standing behind another person. Multi-object tracking [2], [15], [16] can be used to ensure that each object is tracked individually even if it is not visible in one frame. The tracking also falls apart when people enter/exit the camera view at the same time or when there are multiple occlusions in the scene. A second challenge is to ensure that something is in place. The number of people in the area that a camera can see could mask a situation in a small area. A door may fill up very rapidly, while an open area, which is close by, remains relatively empty. Checking the number of people in areas and, within set limits [4] is therefore helpful because the system can make decisions about a place that has an real meaning. The concept behind CrowdGuard AI involves analyzing the space under observation as the unit to determine the number of people and establish limits.

1.3 Problem Statement

This is a problem related study, as there is no practical monitoring system to link visual crowd analysis, independent sensors, zone-specific limits and automatic responses in the temple-festival context.Data is available from a camera system, but can be inaccurate if lighting is poor, people block the view or the view is blocked.A beam counter can count the number of people going

through a point. It doesn't provide information on the location of people over a larger geographical area.

So the research question is: How can a light weight AI and IoT system estimate the number of people, in a high risk area, verify the number with a sensor, track the change in occupancy over time, and then send a warning if a safety limit is exceeded? The goal is to not predict a stampede. The goal is to create an early warning system that will transform crowd conditions into warnings.

1.4 Motivation

The primary motivation for this effort is to transition from 'watch' mode to 'early warning' with measurable outcomes. A camera feed isn't really too useful. However, when that same feed is converted into data over time, like how many people are there, how dense is the crowd, how fast does the crowd increase, and whether the crowd is approaching a maximum – the system is much more useful. This assists us to determine whether there is a calm crowd or whether the crowd is rapidly increasing in size in a particular area. Systems running on edge devices also indicate that lightweight models are necessary to accommodate limited hardware and speedy responses to the systems [7].

Redundancy is a second. If two or more sensors fail using than one sensor can be used to help prevent errors. Data can be provided by a camera and can display people in a particular zone. However, a beam sensor on the hand can only let you know that someone crossed a point. With this, the system can also know if one sensor is sending a signal for only a single reading. In cases such as lighting or when people obstruct the field of view, cameras may not function properly [20] which can be useful in such instances.

The third motivation is that the goods are affordable. Temple trusts, local event organizers, and small control rooms may not have access to expensive multi-camera analytics systems. The proposed prototype uses off-the-shelf components such as ESP32, ESP32-CAM, a beam sensor, buzzer, LED, and a laptop/PC, with lightweight software components that can be adapted to different locations.

1.5 Research Objectives

The research objectives are: (1) develop a real-time crowd-density monitoring prototype for temple festivals and related gatherings; (2) detect people using AI-based computer vision; (3) maintain person identities across frames using ByteTrack; (4) calculate occupancy within a defined zone; (5) obtain an independent crossing signal using an IR/laser break-beam sensor; (6) combine the two sources through sensor fusion; (7) calculate crowd density and growth rate; (8) apply a zone-specific occupancy threshold; (9) classify the monitored condition using warning levels; and (10) trigger a local alert when the configured critical condition is reached.

1.6 Contributions of the Proposed Work

The first contribution is the integration of object detection and multi-object tracking into an occupancy-focused crowd-monitoring pipeline. YOLOv8n provides the person detections required by the prototype, while ByteTrack maintains identities across frames. The second contribution is the use of a separate IR/laser crossing sensor as an independent input rather than treating the camera as the only source of occupancy information.

The third contribution is zone-aware decision logic. Instead of applying one global threshold to an entire scene, the system can define a monitoring region and associate the region with a physical area and a configured capacity. The fourth contribution is trend warning logic: the current count is taken into account along with the rate of change which helps the prototype tell the difference between a steady situation and one that is increasing quickly.

The fifth contribution is an IoT alert path. The ESP32 can take sensor input. Control a buzzer and LED while the computer-vision dashboard shows the count, density, laser events, threshold, FPS and system status. The present contribution is therefore an integrated prototype architecture and functional evaluation of a statistically complete large-event validation study.

TABLE I. RESEARCH OBJECTIVES AND VERIFICATION PATH

Objective	Implementation Element	Verification
Real-time monitoring	Camera + OpenCV	Continuous frame processing
Person detection	YOLOv8n	Bounding boxes
Tracking	ByteTrack	Stable track IDs
Sensor support	IR/Laser + ESP32	Crossing events
Zone occupancy	ROI / zone logic	Count within monitored region
Risk state	Threshold + growth rate	Green/Yellow/Orange /Red
Local response	Buzzer + LED	Alert activation
Dashboard	Live application	Count, density, status, FPS

II. LITERATURE REVIEW

2.1 Traditional Crowd Monitoring Methods

Manual observation is still the way to manage crowds. Security staff can spot lines, blocked doors, strange movements and calls for help. The advantage of observation is knowing the context: human workers can use their understanding of the place and the current situation. However manual observation is hard to expand when many doors and tight spots need to be watched at the time. People working with cameras can also get overwhelmed with too much information.

CCTV monitoring improves the view by giving video. It helps with looking at events and watching things directly but a normal CCTV setup usually leaves it up to an operator to figure out the risks. The benefit of AI-based analysis is that the video can be turned into numbers and data that a machine can understand. Learning crowd monitoring has repeatedly shown that it is better to move from watching videos by hand to using analysis [5] [6] [22]. This will automate the process to make it faster, more consistent and less reliant on attention.

RFID-based counting can provide accurate passage information in controlled environments where people carry tags. Such a design can be useful for ticketed venues or access-controlled facilities, but it is less suitable for open festival environments where large numbers of visitors are not expected to carry a common identifier. This makes passive sensing attractive for a public gathering prototype.

IR or laser break-beam sensing provides that passive alternative. When a person interrupts the beam, the controller receives a discrete event. The method is inexpensive and easy to understand, but it is spatially local: it tells the system that a crossing occurred at one point. It cannot, by itself, determine whether a broader area is crowded. CrowdGuard AI therefore treats the beam sensor as a complementary input rather than a replacement for camera analytics.

2.2 AI-Based Crowd Density Estimation

CNN-based crowd counting approaches learn a mapping from an image to a density map. Early deep models demonstrated that high-level semantic features and low-level spatial cues can be combined to handle the scale variation found in crowd scenes. CrowdNet, for example, combined deep and shallow convolutional features and used multi-scale augmentation to improve dense-crowd estimation [9]. Such methods illustrate why crowd counting cannot always be treated as ordinary object detection.

Multi-column CNN approaches further address scale by using filters with different receptive fields. The MCNN model was designed for

arbitrary crowd density and perspective, using multiple columns to capture different head or person scales [10]. This idea is useful when an image contains both near and distant people. However, density-map methods normally produce aggregate information rather than explicit identities that can be tracked from frame to frame.

CSRNet used convolution in the back end of a density estimation network keeping the picture details while making the area it looks at bigger [3]. Its performance on pictures showed how useful density maps are for very packed places. In a setup like a temple gate though the system also needs a decision process, hardware that works together and alerts that are close by. So CrowdGuard AI uses a detector and tracker for the current model and keeps dense-crowd regression models as options, for when there is extreme crowding.

The research also shows how important it is to focus on an area. Pardamean et al. Created a learning method for counting people inside an area without just covering the image [4]. This makes using an area in the current system a good idea. In life a temple entrance can be a shape with straight lines so the number of people matches the space that has a safety limit.

YOLO created a way to find objects in one step predicting boxes and chances of what's in the image, which is good for quick use [13]. Later versions of YOLO made improvements in how good they're how fast they work and how easy they are to use. Reviews of the YOLO group show the balance between how big the model's how fast it works and how well it finds things [14]. CrowdGuard AI uses YOLOv8n because the initial version needs a model and the real setup already uses the Ultralytics YOLOv8n system.

2.2.1 CNN-Based Density Estimation

CNN for counting people is usually a problem where the network creates a map that shows where people are and the total in that map is the estimate. The benefit of this method is that this method shows where people are. The problem is that training needs maps with people marked or places where heads are and training can take longer than a model. Research shows that hiding people, small areas changes in size and changes in appearance are problems for this method [12].

2.2.2 YOLO-Based Person Detection

Single-step models are good for monitoring because single-step models can find things without a step to locate possible areas. YOLO started this way and later versions of YOLO made better the balance between speed and how well they find things [13] [14]. For the system the main thing is not how good it is compared to others but how fast it can find people and pass that to ByteTrack in a loop.

2.2.3 Tracking-Based Crowd Monitoring

Adding time to finding people helps. Of looking at each picture alone a tracker joins the finds so one person has the same identity over many pictures. ByteTrack is used in the system because it connects the boxes and keeps useful tracks even if some finds are not sure [2]. Other trackers like Deep SORT use how people look to make the identity stay the same [16]

2.3 IoT-Based Crowd Monitoring Systems

IoT setups take crowd monitoring beyond the camera by linking sensors, controllers, ways to send data and warning devices. The real benefit is that parts can be separate: a sensor can be near a door while the main finding runs on a computer. The ESP32 group offers data and inputs making it good for a low-cost first version.

Recent studies about IoT-based crowd monitoring use cameras with small models and a monitoring computer.The YOLOCROWD/YOLO-GHOST work, by Rethnasamy et al. Shows a setup that sends video segments for detection and counting before sending results to a monitoring location [19]. Such designs show the usefulness of combining vision with an infrastructure while also highlighting the importance of inference speed and communication.

Edge-intelligence work on crowd density estimation also emphasizes that resource constraints can make a large deep model impractical for continuous monitoring. Wang et al. showed how a lightweight CNN could be adapted for edge deployment while retaining useful density-estimation performance [7]. This supports the choice of a lightweight YOLOv8n configuration for the current prototype.

2.3.1 Edge Computing Considerations

Edge-oriented crowd analytics reduces the need to transmit every raw frame to a remote server. This can lower bandwidth use and reduce the latency associated with remote inference. The edge-intelligence literature also points out the trade-off between model size, inference speed, and estimation quality [7]. CrowdGuard AI follows a hybrid edge approach in which video processing is local to the laptop/PC, while the ESP32 performs sensing and local response.

2.3.2 IoT Communication and Alerting

An IoT monitoring system becomes operationally useful when sensor measurements can be converted into events that reach the responsible operator. I believe the communication design should define message structure, time synchronization, acknowledgement, retry behaviour and failure handling. For the prototype, buzzer and LED outputs are deliberately simple so that the alert path remains available even when remote communication is unavailable.

2.4 Sensor Fusion Techniques

Sensor fusion can be defined as the process of combining measurements from two or more sensing modalities to support a decision. In the system the modalities are heterogeneous: a camera returns structured visual detections while an IR/laser beam returns discrete crossing events. Fusion can therefore occur at levels, including raw-signal fusion, feature fusion or decision-level fusion. CrowdGuard AI implements a decision-oriented approach that compares the camera count with the sensor count before the risk decision.

The simplest useful fusion operation is consistency checking. Suppose the camera estimates C camera-based occupants or crossings and the laser sensor records L events. A large discrepancy $|C - L|$ may indicate occlusion, double counting, sensor misalignment or a difference in what each source's measuring. The system should not automatically add C and L because that would double-count the people. Instead the difference is treated as a reliability indicator. The measurements are used to support the occupancy decision.

Multi-sensor fusion becomes particularly useful when failure conditions are different. A camera can be degraded by darkness or occlusion whereas a beam sensor is less dependent on light texture but only observes a narrow crossing line. The proposed architecture therefore seeks complementarity than redundancy for its own sake.

2.4.1 Fusion Levels

Sensor fusion can be grouped into data-level feature-level and decision-level fusion. Data-level fusion combines measurements before interpretation. Feature-level fusion combines derived attributes. Decision-level fusion combines decisions or confidence states after each sensor has been interpreted. The current CrowdGuard AI design uses decision-level fusion because the camera and beam sensor have data types and sampling behaviour.

2.4.2 Reliability and Sensor Disagreement

The system can maintain a sensor-status flag alongside the occupancy estimate. When the discrepancy between camera and sensor events becomes large the system can raise a condition. This is preferable to forcing both sources into an arithmetic count. The approach also allows future calibration data to be used for weighting or rejecting a sensor temporarily.

2.5 Research Gaps

The literature suggests five gaps that're directly relevant to the proposed work. First many crowd-counting studies focus on image or video benchmark datasets than the practical integration of sensors

and alarms at a specific deployment site. Second several systems are designed around a visual sensing modality. Third aggregate scene counts do not always represent congestion, motivating ROI or zone-aware monitoring[4].

Fourth early warning is often described in terms of density or abnormal motion. Is not necessarily connected to a low-cost physical alert path. Fifth, validation at the scale of a temple festival remains challenging because open venues have lighting, non-uniform perspective changing camera placement and complex crowd behavior. Dense-Stream YOLOv8n demonstrates the potential of real-time detection but its primary experimental setting is smart-library visitor flow [1]. The proposed system therefore focuses on the integration of camera, sensor, thresholds and local alerts rather than claiming a universal model advantage.

The research gap can be summarized as the need for a pipeline, in which visual detection, tracking, zone occupancy, independent sensing, trend analysis and alert generation form one operational loop. The current prototype is designed around this sequence and provides a preliminary functional demonstration of the complete loop.

TABLE II. LITERATURE COMPARISON OF RELEVANT APPROACHES

Study	Primary Technique	Deployment Focus	Observed Limitation
Dense-Stream YOLOv8n [1]	YOLOv8n + DensityNet	Smart library / edge	No multi-sensor fusion
ByteTrack [2]	Detection association	Video tracking	Not a density or alert model
CSRNet [3]	Dilated CNN density map	Dense crowds	No direct tracking / alert path
ROI counting [4]	Deep learning + ROI	CCTV regions	Requires ROI-specific learning strategy
YOLO edge [5]	YOLO-based detector	Resource-limited edge	Detection-focused
YOLO-CROWD [19]	YOLO-GHOST + IoT	Networked surveillance	Depends on video transmission
Edge CNN [7]	Lightweight CNN	IoT / edge	Density-model specific
CrowdGuard AI	YOLOv8n + ByteTrack + IR/Laser	Temple-festival prototype	Broader validation still needed

III. PROPOSED CROWDGUARD AI FRAMEWORK

3.1 Overall System Architecture

There are four layers of the system: sensing, perception, decision and response. The data is collected in the sensing layer where a camera and an IR/laser sensor are used. The perception layer deals with the image preprocessing for YOLOv8n (frame preprocessing for detection) and person reid (person tracking over time) with ByteTrack. The decision layer provides calculations for zone occupancy, zone density, growth rate, sensor agreement and sensor threshold status. The response layer triggers alerts. Pushes the state of the current system to the monitoring dashboard.

This design de-couples processor operations from hardware input-output operations. The video processing pipeline is performed by a laptop or PC and the ESP32 controls sensor inputs and output devices. This partitioning helps to decrease the burden on the microcontroller. It also allows for the power of the CPU and GPU to be used by the computer vision model without the need of a high end edge accelerator in the prototype.

The architecture is of a modular type. Later, if we wish to switch to a detector, the tracking layer and the decision layer can remain virtually unchanged. We could add another sensor at a gate, assign it to a zone and continue to operate the system without issue. This renders the design suitable for deployments at chokepoints, one monitoring node .

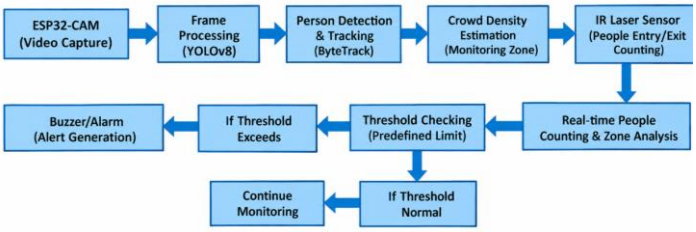


Fig.1.Larger picture of the CrowdGuard AI architecture, which features the ESP32 camera sensor node, sensor fusion, zone-thresholding and local response path.

3.2 Working Principle

First working principle is that a frame is captured by the camera. YOLOv8n is fed with the frame, and it outputs the bounding boxes and confidence scores of the detected persons. These detections are sent to ByteTrack where they are connected to people over frames. This generates a list of objects being tracked, along with their positions, that can be compared to the monitoring zone.

When the ESP32 receives data from the IR/laser sensor. When a beam is broken it generates a sensor event. Increases the number of crossings. By default, the camera count and the laser count are not considered the thing. Instead their relationship is analyzed during the fusion step. The system then calculates the occupancy and density that are required for the safety decision.

The decision module determines the occupancy level, the threshold as well as the rate of change. When everything is normal monitoring continues. If the system reaches a warning state, the status on the dashboard will increase. When the critical condition is met the response layer turns on the buzzer and the LED. Also preserves the event.

3.3 Multi-Sensor Data Fusion

A measure of agreement can be used to describe a practical fusion rule. The number of sensor events during the observation window is denoted by L, and the number of cameras is denoted by C. The discrepancy is defined as E = |C − L| where C is the measurement and L is the law. This is not an accurate universal measurement since the camera and laser could be taking in portions of the scene. Rather it is used as a trigger for consistency check. When E is unexpectedly high the operator sees a sensor-disagreement state or a silent count.

$$E = |C - L|$$

- **E** – Sensor discrepancy
- **C** – Camera-based count
- **L** – IR/Laser sensor event count

One such consideration is time matching. Camera count: number of people currently in the area. The break-beam sensor highlights to people who walked through a location at a particular time. The system compares the two with a time period. For such an arrangement as a sensor system, entry/exit beams may be installed at both ends of a hallway. This would allow the system to determine the number of people in the system by subtracting Entry from Exit. It compares that number with the one that the camera sees.

The current model keeps the combination logic simple so it can be explained, checked and tested while it is being built. A better version could figure out how reliable each sensor is, by looking at disagreements. This kind of model needs data to work. Is not part of what is being tested now.

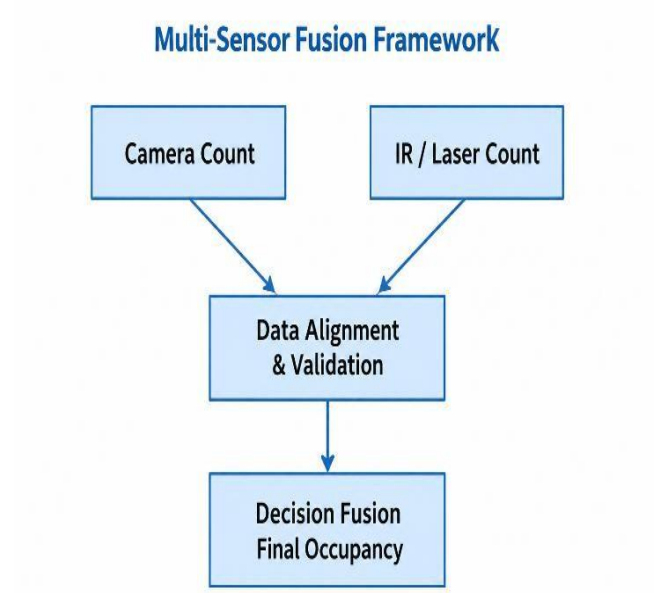


Fig. 2 Adding up the number of people on the camera and IR/laser crossing data.

3.4.SystemWorkflow:

The system process can be broken down into the following steps: initialization, data collection, analysis, combination, decision, action and review. Initialization begins with loading the model, turning on the camera, prepping the ESP32, configuring the laser counter and establishing a limit. Data collection is ongoing for every picture as it is received.

Analysis part consists of processes such as generation of the image, detection of people and people of interest, and counting. The area is arranged so that the center of the person is only counted if the space criterion is met. This can prevent double counting of people not in the main area. The number of persons/meter is determined by dividing the number of persons by the area.

The beam status is controlled by the sensor part. The sensor is updated to reflect the number of people passing through. The two paths merge in the combination part. The system subsequently compares the limit and risk level. The process repeats for the picture or time periods. The review step is crucial as it is a job and not just one decision.

The observer points out that the workflow maintains synchronization between the camera processing path and sensor monitoring path. The camera provides data on the occupants of the monitored area, and the laser sensor marks crossing events. These inputs are added to the threshold. Growth-rate data to support the risk decision. Once the alert/monitoring state is set, the system enters into the processing cycle and continues monitoring the zone.

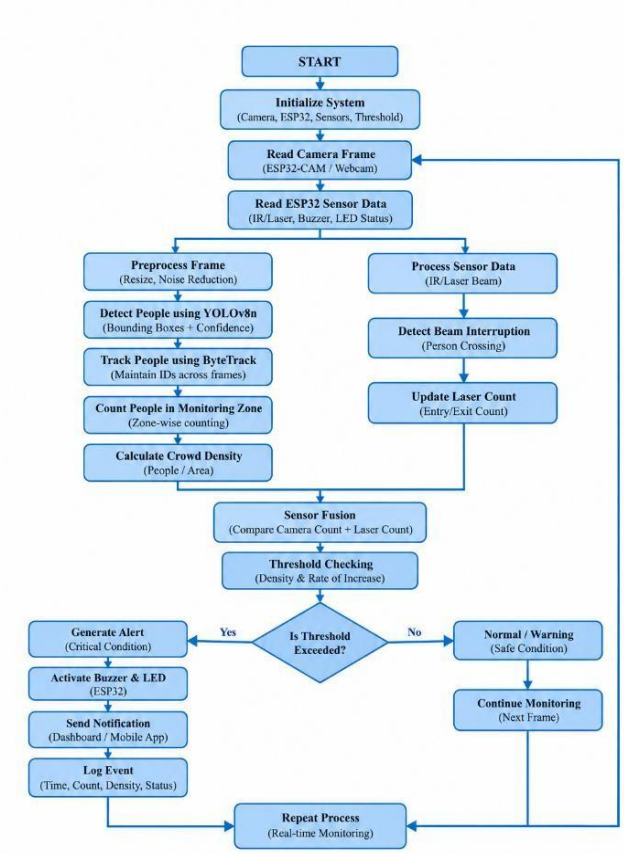


Fig. 3. System workflow starts with system initiation and sensing, and then progresses to alerting based on thresholds.

3.5 Alert Generation Strategy

The alert strategy is based on a hierarchy of a yes/no trigger. When the system shows a light the system indicates that occupancy is still inside the range. If the light is yellow, then it's nearing full capacity. An orange light means that there are a lot of people or people are filling up quickly. A red light means that a critical level has been exceeded. The colors aid operators in understanding the system state's meaning at every time.

The hardware that responds now is local. A buzzer along with an LED controlled by an ESP32 provides the warning. The people count, density, density rate, laser count, threshold, processing FPS and overall system status is displayed on one dashboard. Useful for prototype testing because the developer will be able to see the numbers and the physical alert at the same time.

This is not considered a prototype feature, but an add-on. An alert in a production system might be directed to a web endpoint, a control-room dashboard, an SMS gateway or a messaging service. All of those connections would have to be verified, for reliability and privacy.

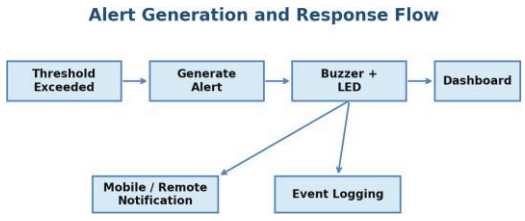


Fig. 4. Local and remote generation path for alerts.

3.6 Advantages of Proposed System

The framework has the following six benefits. First it relies on its components. Second it gives real-time detection and tracking with a detector. Thirdly there is a physical sensing channel added. Thirdly, it allows each zone to choose its thresholds. Fifth it associates an analysis of the software with an alert. Sixth is able to be expanded to incorporate cameras or additional sensor nodes.

These are the benefits and do not constitute proof that this system is superior to any others. The current test reveals the interaction and operation of the parts in a controlled prototype environment. There must be more tests of the size of the crowd and the settings to measure the actual performance improvements, in alerting.

IV. HARDWARE COMPONENTS

4.1 ESP32 Controller

The ESP32 is used as the controller of the sensing and alert. It monitors the status of the beam-sensor and controls the buzzer and lights. If it is used to network, with a future version, then it may send state data to the processing system without requiring another microcontroller board.

The design of a good controller should separate the sensing, communication and output handling in software. Beam events may be de-bounced or time filtered to avoid being counted more than once in case of signal noise. The controller has the option of adding a time stamp to each event before sending it to the monitoring process.

4.2 Camera Module

The camera is the sensor that is used to detect activity. The project plan calls for an ESP32-CAM as the embedded camera to be used, and a standard webcam for testing purposes during development. The camera should be mounted to ensure that the area to be monitored can be seen at all times. It should not be exposed to light sources, strong backlighting or locations where a lot of moving objects obscure the light.

The camera used will determine the amount of work the system must undertake. The higher the resolution of a camera, the more detail it provides but it requires more processing power. For a real-time prototype using a moderate resolution balances image quality and frame rate well. Future installations may require the use of fixed focus cameras, wider angles than one camera (if one camera is not sufficient to cover the entire chokepoint area).

4.3 IR/Laser Break Beam Sensor

When the beam is broken, the sensor will be activated. In the prototype the beam runs across a narrow path or test line. The sensor must be oriented properly to ensure that when the target is moving normally, it will be detected, but when it is moving slowly, it will not trigger the sensor again and again.

Adjustment is needed for proper placement. The beam might be too high so people will walk under it without activating the sensor. If it's too sensitive, bags or small items might trigger it. Several beams may be used to establish a direction or to increase the reliability. The present prototype is just used for basic functionality using one beam.

4.4 Wi-Fi Communication

Wi-Fi enables the embedded system to communicate with a computer or dashboard. The ESP32 can send the data measured by the sensor or the event message to the monitoring app. In a real-world situation the connection needs to be secure – authenticated and encrypted. There should be rules in the system regarding the consequences of network failure.

If local alerts are sent, they should not be interrupted by losing Wi-Fi (for safety-monitoring purposes). When communication is lost, the buzzer and LED will serve as indicators, while when the network is operating, the remote dashboard will provide additional visibility features.

4.5 Power Supply

A stable power source is required for the ESP32, the camera, the beam sensor and the alert devices. Breadboard wiring is sufficient for testing, but is unsuitable for use in the field unless wires are solid connectors or enclosed in an appropriate housing. Calculate the power, considering the camera, wireless transmission and indication lights and any sudden power surge.

Power should also be taken into account for future deployments. The monitor will continue to function in the event of a power outage thanks to a UPS or battery system. This is significant due to the fact that if an alert system restarts if the power goes down, it could be reset and calibrated.

4.6 Cloud/Local Server

Laptop/PC is the main processing server for the current prototype. This is logical during the development process because the model loading, image analysis, logging and dashboard can be carried out on the device. Testing without a server also results in reduced dependency on the internet.

n production edge processing could run separately from storage. A processing unit may be located within each zone, and only exchange summary data—such as count, density, risk level, and sensor agreement data—with a central dashboard. This cuts down on raw video traffic.Improve privacy.

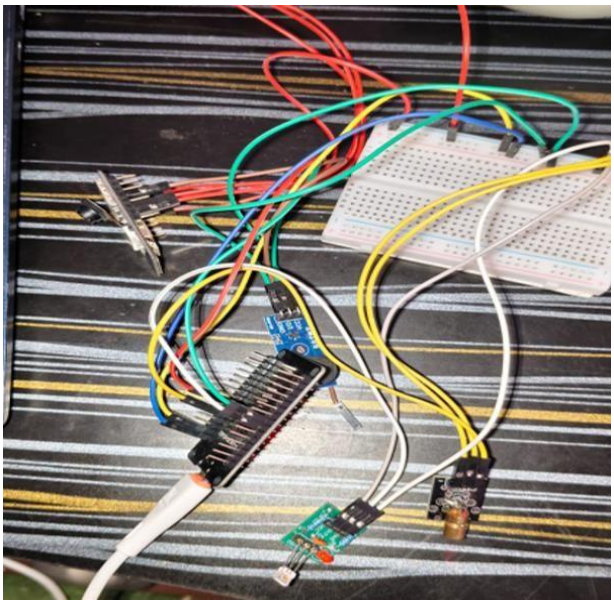


Fig. 5. Showing physical prototype assembly with ESP32 controller, breadboard, sensor modules and jumper connections.

TABLE III. HARDWARE SPECIFICATIONS USED IN THE PROTOTYPE

Component	Model / Type	Role
Processing unit	Laptop / PC	AI inference and dashboard
Camera	ESP32-CAM / Webcam	Video acquisition
Controller	ESP32	Sensor and alert control
Beam sensor	IR / Laser	Crossing event detection
Alert output	Buzzer	Audible warning
Status output	LED	Visual warning
Prototype board	Breadboard	Circuit interconnection
Wiring	Jumper wires	Signal and power connections

TABLE IV. SENSOR CONFIGURATION

Sensor / Signal	Measured Event	Use in Decision
Camera	Detected person / track	Zone count and density
IR / Laser	Beam interruption	Crossing-event count
ESP32 status	Sensor / communication state	Node health
Buzzer / LED	Alert state	Local operator indication

V. SOFTWARE METHODOLOGY

5.1 Image Acquisition

The camera stream is used for image acquisition. Converts it into frames which can be processed by the AI. OpenCV helps to read stream, do processing on the image buffers. The processing loop needs to keep a frame rate and not copy big image arrays unnecessarily.

The image may be scaled down, modified to change colors, and possibly filtered to remove noise during the pre-processing stage. The aim is to create minimal changes to the scene and ensure that the input data remains consistent for YOLOv8n. The same procedure should be used for testing so that the FPS measured is real.

TABLE V. SOFTWARE ENVIRONMENT

Software	Version / Type	Purpose
Python	3.12	Application logic
OpenCV	5.0.0.93	Video and image processing
Ultralytics	8.4.104	YOLO implementation
YOLOv8n	Nano model	Person detection
ByteTrack	MOT algorithm	Person tracking
NumPy	2.4.6	Numerical operations
PyTorch	2.13.0	Deep learning runtime
Torchvision	0.28.0	Vision utilities
VS Code	IDE	Development
Arduino IDE	2.x	ESP32 firmware
Windows	10/11	Operating environment

5.2 Crowd Detection using YOLO

This is something where YOLO is useful, as it considers object detection as one of its tasks of prediction [13].Each pass forward may yield several crowd detections, which is beneficial for ongoing crowd monitoring. The original YOLO work set a paradigm for real-time detection, while later versions tackled the speed vs accuracy trade-off better.

The present prototype uses YOLOv8n because it is a lightweight model in the YOLOv8 family and is available as part of the Ultralytics software stack. The model outputs bounding boxes and confidence scores for the people in every frame. A confidence threshold can be set to determine the balance between false detections and missed detections.

YOLOv11 is a newer model that can be used in future experiments, but it is not the model that is used in the reported experiment. In this way, the current paper is not confused with the expected results and not mixed with upgrades to the actual YOLOv8n. Future work could involve comparative testing of YOLOv8n with a newer, lighter and detection algorithm.

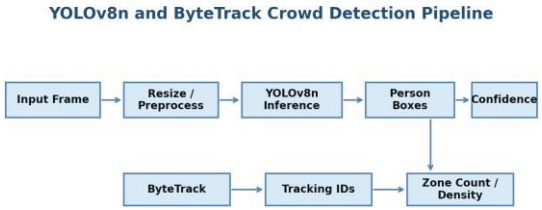


Fig. 6. The prototype adopts the following detection and tracking pipeline: YOLOv8n Detection + ByteTrack Tracking.

Why YOLO?

In one-stage detector, the selection has been driven by the desire to update the video stream. A detector that requires steps may make things slower and more difficult to use. Object Detection in a single step is done by YOLO. Was designed for quick processing [13].

Model Selection (YOLOv8n / YOLOv11)

Now it's using YOLOv8n, which is light and already supports the project software. The speculations for YOLOv11 are not about testing something now, but rather something to compare in the future. This option allows the working version of the paper to be revealed, and adds a buffer zone to future work**.**

5.3 Crowd Density Estimation

Estimation Crowd density: The number of people in the area divided by the area of the crowd. If N is how many people are seen and A is the area in meters density is D = N/A. The current demo had N = 1 and A giving D.

$$D = N/A$$

- **D** – Crowd density
- **N** – Number of detected people
- **A** – Monitored area zone in meters

The area can be a rectangle or a polygon. A polygon is better, for doors and hallways, as it can be shaped to the floor. Each zone can also store its own area, safe occupancy threshold, warning threshold, and sensor association.

Density alone does not fully describe crowd risk. Two zones with the same count can have different crowding characteristics because their available areas differ. Therefore, density and occupancy are kept as separate variables in the decision logic.

5.4 Person Counting

Person counting is performed from tracked detections rather than simply adding the number of raw detections in every frame. ByteTrack associates detections across frames and maintains track identities. The count can then be based on the number of active tracks whose reference point lies inside the selected zone.

Tracking is important because a moving person may appear in many consecutive frames. Without temporal association, each frame would contain a fresh detection and the system could easily double-count. ByteTrack was designed to associate detections across frames and recover useful tracking information even when some detections have lower confidence [2].

Counting performance can be expressed using a simple absolute counting error, $E_{count} = |C_{detected} - C_{actual}|$. In the current single-person demonstration, this error is zero because the system observed one person and detected one person. This observation applies to the scene that was shown and does not represent an estimate of accuracy, across the entire population.

$$E_count = |C_detected - C_actual|$$

- **E_count** – Absolute counting error
- **C_detected** – Number of people detected by the system
- **C_actual** – Actual/ground-truth number of people

5.5 Sensor Fusion Algorithm

The fusion step takes a count from the camera and a count from the laser sensor Compares two things over a specific period of time to determine if they are different or if they are the same. The system uses rule-based conditions so that every alert can be traced back to its source inputs.

A simple decision function is defined as F(C, L, T, G). Here C stands for the camera count L for the laser-event count or sensor state T is the threshold value and G represents the density growth rate. This function returns the monitoring state of the system. In a safety-focused design any disagreement between sensors should not be ignored. Instead it should trigger an alert so that operators know something is wrong.

The current prototype uses a rule-based fusion method than a machine learning model. A learned fusion approach would need lots of labeled data from environments, which makes debugging more difficult. Rule-based fusion is easier to explain and verify in stages because each decision comes from known input values making the logic transparent.

TABLE VI. COMPUTATIONAL AND DECISION VARIABLES

Variable	Definition	Current Use
C	Camera-based count	Current zone occupancy
L	Laser events	Independent crossing signal
A	Zone area (m^2)	Density denominator
D	C / A	Crowd density
G	$(C_t-C_\{t-\Delta t\})/\Delta t$	Growth rate
T	Zone threshold	Critical decision
S	Risk state	Green/Yellow/Orange /Red

VI. CROWD RISK ESTIMATION AND EARLY WARNING

6.1 Safe Occupancy Threshold

Safe occupancy thresholding turns real-world safety rules into software logic. A threshold sets the number of people allowed in a zone. This limit depends on factors like the area size, how wide the passage is, the expected flow of movement and other site-specific safety requirements. In the classroom example the threshold was set to one person. It was decided to make that choice in order to test the alert path and not to simulate capacity at a temple.

The threshold should be determined by the safety or facility team of the site and documented for each zone for deployment. This threshold as well as the zone information should be kept in the system so when the warning occurs, the operator is able to interpret what it is.

6.2 Density Growth Rate

The rate of entry to a zone provides additional information. Let C_t represent the number of people at time t and $C_\{t-\Delta t\}$ represent the number of people at an earlier time. Δt is the amount of time in minutes between the two measurements. A simple growth rate is $G = (C_t - C_(t-\Delta t)) / \Delta t$. Positive and large Growth Rate G indicates fast increasing crowd.

This is an important signal because the signal might cross the threshold soon although the current count is below. The normalized occupancy ratio (C/T) and growth rate (G) are two key elements that need to be combined in a good warning system. Adjustments to the weights and alert triggers should be made according to testing at each venue.

In the demo run, the growth rate of the density was 0.574 per minute on the Dashboard. Since this number came from one prototype test it is not considered a fully validated parameter. Rather it is reported as a value obtained from the current run.

$$G = (C_t - C_(t-\Delta t)) / \Delta t$$

- **G** – Crowd-density growth rate
- $\mathbf{C_t}$ – Number of people at time t
- $\mathbf{C_t}$-$\mathbf{\Delta_t}$ – people before the time interval
- **Δt** – Time interval between the two measurements

6.3 Congestion Detection

The congestion detection relies on occupancy and trend information. A zone is considered to be stable if the population size is small and the rate of increase is near zero. It activates a warning when it is approaching the set limit in terms of the occupancy, and/or increasing in terms of the growth rate. A critical condition occurs when the conditions are met.

Movement-flow information can be added to improve the system. If you consider optical-flow magnitude, you can see different changes in movement, such as track and directed counts. These signals will help the threshold-based logic. These are not part of the validated prototype, but are to be viewed as future improvements.

6.4 Prediction Algorithm

The prediction algorithm in the prototype might be better considered as a threshold-based risk estimation and early warning. It is based upon occupancy, density, growth rate and sensor data to decide the current state. It is not intended to forecast the timing and location of a future stampede event.

The steps are detailed below in Algorithm 1: Read count Calculate density Compute growth rate Get laser state Compare with zone threshold Classify laser state and trigger required response. This makes it easy to follow and is useful for a prototype as you can see each decision and confirm it.

Algorithm 1. Threshold-Based Crowd Risk Estimation:

• Read the camera frame, and the state of the ESP32 sensor..

• Identify persons with YOLOv8n. Make detections link with ByteTrack.

• Count the number of tracks within the monitored area.

• Calculate density using the count and the zone area.

• Count and zone area to determine densit.

• Update the time window growth rate

• Compare occupancy and growth rate, with the set zone threshold.

• Once the critical condition is reached, press the buzzer or the LED and record the incident, if it is not reached, continue to monitor.

6.5 Warning Levels

There are four warning levels: Green, Yellow, Orange and Red. GREEN = occupied and stable.YELLOW indicates either occupancy or growth is nearing the warning level.Orange: Occupancy (fast growth) = congestion developing. Red indicates either that the threshold has been exceeded or that a critical condition has been set.

Color names are labels that are used in the operation. These have to be correlated with actions which the event-management team has agreed. These can involve minimising access, redirecting a queue, opening a gate or sending employees. The paper does not state that the thresholds that are presented in the paper are applicable to all temple environments.

TABLE VII. WARNING LEVEL DEFINITIONS

Level	Condition	System Response
Green	Occupancy below warning range; stable trend	Continue monitoring
Yellow	Approaching threshold or rising trend	Display warning
Orange	High occupancy or rapid growth	Prepare intervention
Red	Configured critical condition reached	Alert + buzzer/LED

VII. SYSTEM IMPLEMENTATION

7.1 Hardware Setup

The hardware prototype consists of connecting the camera ESP32, beam sensor, buzzer and LED using a breadboard. The camera is used to get the visual stream, and the ESP32 gets the beam signal to activate the local alerts. During the demonstrated run the hardware produced the sensor events used by the monitoring application.

The prototype photo should be considered as an engineering proof of concept and not a completed outdoor enclosure. The electronics should be mounted in a weather proof box, connectors should be secured and the laser IR alignment should be mechanically secured for a field node.

7.2 Software Setup

The software environment uses Python 3.12 OpenCV, Ultralytics YOLOv8n, ByteTrack, NumPy, PyTorch, Torchvision, VS Code and Arduino IDE on Windows. It is an application software developed in C++ that is used to run the detection and tracking loop, and it is an Arduino code that manages the hardware layer of the ESP32.

A modular software structure is recommended: camera module, detector module, tracker module, zone module, sensor module, fusion module, decision module and dashboard module. This separation simplifies testing because each component can be validated individually before the integration test.

7.3 Flowchart

The flowchart contains two sensor paths that meet at the fusion stage. The visual path begins at frame capture. Continues through preprocessing, YOLOv8n, ByteTrack, zone filtering and density calculation. The sensor path begins with beam monitoring. Produces a crossing event count. The paths converge at sensor fusion. Then pass through threshold and risk evaluation.

When the threshold is normal the process returns to monitoring. When the condition is critical the system produces an alert activates the buzzer and LED updates the dashboard and records the event before continuing. The flow therefore reflects a loop monitoring system rather, than a one time classification

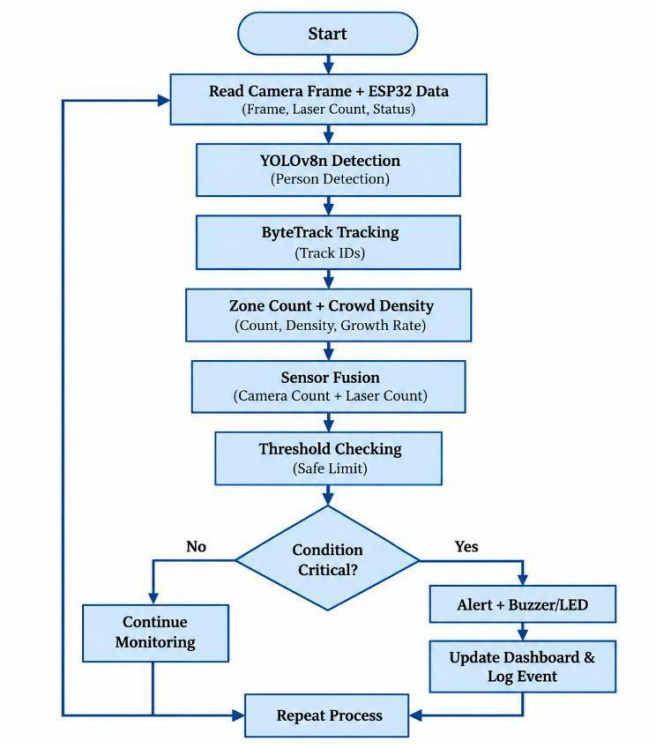


Fig. 7. System flowchart.

When the monitored condition remains within the configured limit the system continues monitoring. When a critical condition is detected the alert mechanism activates the buzzer and LED updates the dashboard and records the event before the monitoring cycle is repeated.

7.4 Communication Architecture

Communication, between the PC and ESP32 can be set up using serial or Wi-Fi transport. The message can contain the recent laser count, event state, timestamp and device status. The PC can then combine the message information with the vision output. I think this approach works well.

A communication layer should also handle packets, device disconnects and duplicate events. A heartbeat or periodic status message can indicate whether the ESP32 is still connected. For a production, authentication and encryption should be done to links.

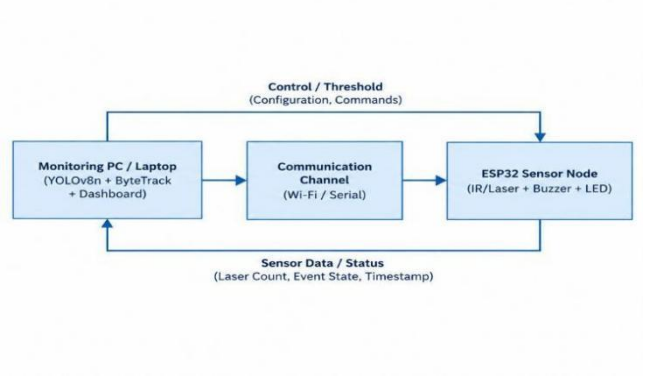


Fig. 8 .Communication Architecture

7.5 Mobile Alert System

The local prototype alert is equipped with a buzzer and an LED. A mobile or remote alert can be added at a later time. This paper claims

to be an extra, unless a particular service has been tested and measured. A secure dashboard can be a means of demonstrating the current status without sending continuous raw video to some remote service.

The zone identifier, current count, density, threshold, risk state, time and sensor status may be present for a future notification event. The message should be brief, giving the operator a quick message to grasp.

VIII. EXPERIMENTAL SETUP

8.1 Dataset

The current paper has two types of data: prototype demonstration data and an extended validation dataset. The results demonstrated in this document are based upon a controlled scene including one person and 100 square meters area. Checking system integration is sufficient but not enough to estimate model accuracy.

The dataset should include videos of the temple or festival with low, medium and high occupancy. Should include camera angles, daylight and low light shots with varying amounts of blocking and various types of movements. To-count density situations are available in public datasets such as UCF-CC50 [10] and the ShanghaiTech dataset [11] that are used for model-level testing.

When training a model, the data need to be divided into training, validation and test sets. If a YOLOv8n model that is already trained is used without changes the experiment should be called inference validation on videos that were not used for training. This is crucial because training and testing on this data would give too good of results.

8.1.1 Temple-Festival Data Requirements

For a temple-festival, the data should include scenes for entrances, narrow paths and open areas where people congregate, in the queue corridors of the temple. Videos should be taken at times and in different lighting conditions. When possible, include the number of people, zone occupancy, and sensor events in the annotations for each video clip. These annotations help with checking accuracy, coverage, counting mistakes and how the system responds to thresholds.

8.1.2 Public Crowd Datasets

The crowd counting methods of UCF-CC-50 and ShanghaiTech are well-known. UCF-CC-50 has crowded scenes while ShanghaiTech has different scene sizes and angles [10] [11]. They are good for testing models even though they do not show the setup of the current temple-festival prototype.

8.2 Experimental Environment

The prototype setup includes a Windows laptop or PC a webcam or ESP32-CAM, an ESP32 controller, an IR or laser sensor, a breadboard, jumper wires, a buzzer and an LED. The computer runs the AI system and the ESP32 handles sensing and alert control. In addition to the classroom demo the project evaluation had real-time test results for detection metrics crowd count changes, density changes, YOLO model comparison, confusion matrix analysis and sensor-fusion response time.

The test used a classroom area of 100 meters and a threshold of one person. The video processing ran at 18-25 FPS, with 23 FPS shown in the dashboard picture and the laser sensor had four detection events. The extended evaluation shows the results for detection metrics and response time under the conditions of the prototype.

The results are explained based on the hardware used the camera view, the test scenes and the testing process. The measurements offer real project proof. Do not say how the system will work everywhere.

TABLE IX. EXPERIMENTAL PARAMETERS

Parameter	Value	Interpretation
Test area	100 m²	Classroom demonstration zone
Detected persons	1	Visible person in test scene
Threshold	1 person	Prototype alert trigger
Density	0.010 persons/m²	Calculated from 1/100
Laser events	4	Beam interruptions observed
Processing speed	18–25 FPS	Demonstrated run
Dashboard FPS	23 FPS	Visible observation
Density rate	0.574 per min	Dashboard observation
Tracking	ByteTrack	Enabled
Sensor fusion	Camera + Laser	Integrated

8.3 Evaluation Metrics

The system was tested with detection, counting, time-based crowd-density, model-comparison and alert-response measures. Accuracy, precision, recall, F1-score and mean absolute error (MAE) were used to check the results of person detection and counting. A confusion matrix was made to see how well the system told people apart from the background. The time-based check also looked at changes in the number of people. Crowd density compared to the set thresholds.

The processing speed was checked with frames per second (FPS) and the alert-response check compared how fast the camera-only laser-sensor- camera-laser sensor-fusion systems worked. The model comparison used person-detection and overall mAP@0.5 results for YOLOv5, YOLOv8n, YOLOv8s and YOLOv8m. The numbers given are from the tests done and are tied to the setup and data used not as benchmarks.

The current paper gives both the test results and the functional prototype observations. The numbers are for the scenes and setup tested; more testing, with repeated clips, clear ground truth, different crowd levels and varied conditions is needed for better results.

TABLE X. PERFORMANCE METRIC DEFINITIONS

Metric	Observed Result	Purpose / Scope
Accuracy	0.92	Overall reported evaluation output
Precision	0.89	False-positive control
Recall	0.93	Missed-person control
F1-score	0.91	Combined detection measure
MAE	0.05	Counting-error indicator
FPS	18–25	Real-time processing speed
Alert response	1.8 / 1.2 / 0.6 s	Camera / Laser / Fusion

IX. RESULTS AND DISCUSSION

9.1 Crowd Detection Results

The real-time detection evaluation showed an accuracy of 0.92, precision of 0.89, recall of 0.93 and F1-score of 0.91 with an MAE

of 0.05 for the reported evaluation output. These numbers show how well the system was able to detect and count people under the conditions that were tested. Figure 9 shows the values and Figure 20 provides the confusion matrix for person and background decisions.

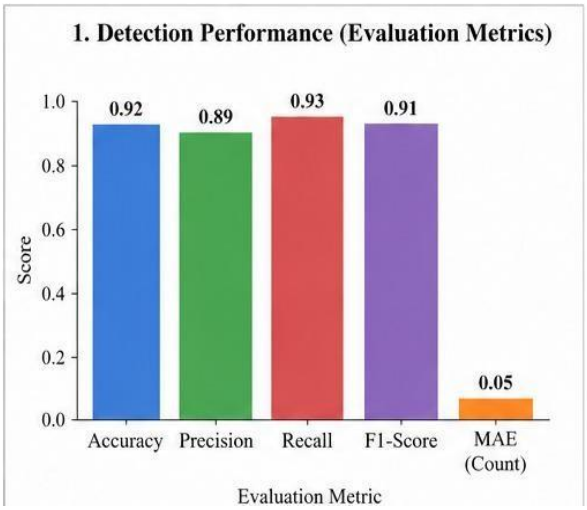


Fig. 9. Detection performance came from the time CrowdGuard AI evaluation output.

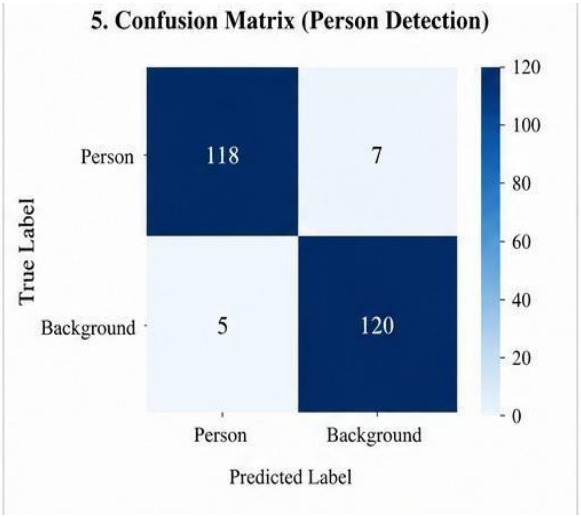


Fig. 10. The fusion result of the person detection evaluation

The prototype had a processing rate that was between 18 and 25 FPS during the real-time runs with 23 FPS shown in the dashboard demonstration. ByteTrack was always on during the monitoring process, which helped connect detections from one frame to the next. The results are for the scenes and setup that were tested and should not be seen as a general statement about the models accuracy.

The earlier single-person test is still useful as a working check: one person was seen the density was calculated the laser path made events and the alert was turned on. That test result is reported separately from the evaluation data and not used as the main model accuracy.



Fig. 11. CrowdGuard AI dashboard showing one person detected, density, density rate, laser count, threshold, FPS and important system status.

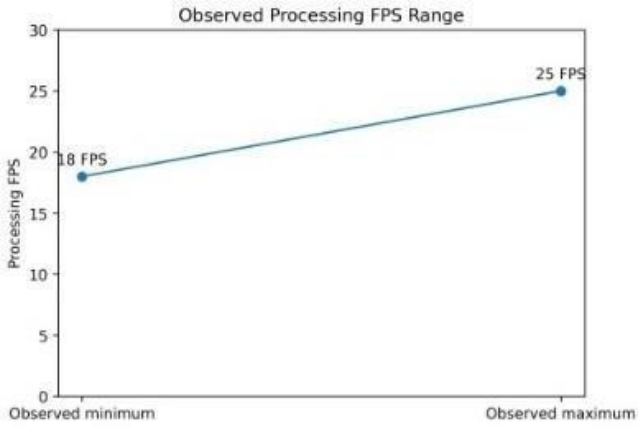


Fig. 12. Observed processing-speed range in the demonstrated prototype run

9.2 Sensor Validation

prototype run. 9.2 Sensor Validation The laser sensor made four detection events during the test showing that the beam-sensor path was working and talking to the ESP32 system. The sensor events were used as an input to the decision part instead of just adding them to the camera count. The camera and laser data are types of measurements so they need to be matched within the same time period.

The longer test also looked at how the sensor system responded. Figure 15 compares camera-only laser-sensor and combined sensor-fusion setups. Controlled crossing tests are still good, for measuring event sensitivity and false triggers in the future.

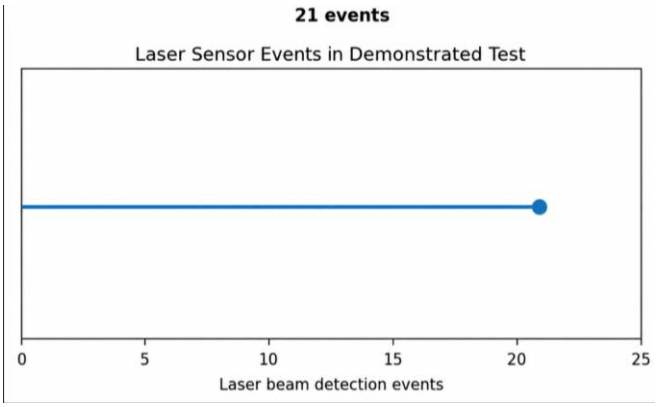


Fig. 13. Laser sensor events recorded during the test of the prototype.

9.3 Density Prediction Results

A real-time time-series evaluation was used to observe how detected occupancy changed over time. Figure 14 shows the detected people count plotted against time along with the configured threshold. In this scenario the count started from a level rose over time crossed the threshold near the middle of the test triggered the alert condition and then decreased as the scene became less crowded.

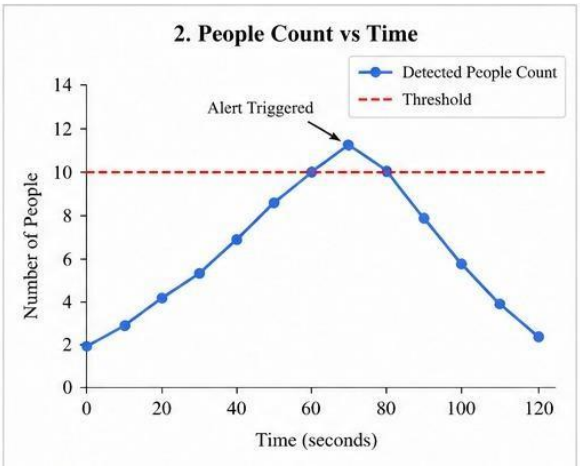


Fig. 14. Detected people count versus time with the configured alert threshold..

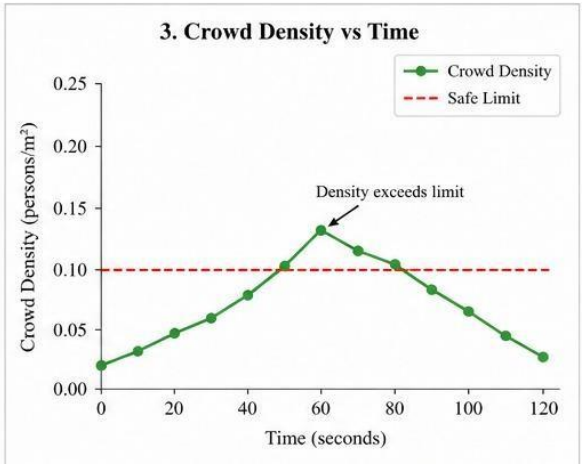


Fig. 15. Crowd-density variation over time to the configured safe limit.

Fig. 15 shows the corresponding variation in crowd density. The density. Went above the configured safe-limit line before decreasing again as the number of people in the area dropped. This time-based behavior highlights the operation of the trend- monitoring layer more clearly than a single snapshot of occupancy. The density is calculated using the formula D = C/A, where C's the number of people and A is the area of the monitored zone. However the threshold and growth-rate values still need to be set based on the characteristics of each deployment zone.

The dashboard also reported a density-rate value of 0.574 per minute during the functional demonstration. This value is an output from the dashboard and should not be treated as a universal model parameter.

$$D = C / A$$

- **D** = Crowd density
- **C** = Number of people detected
- **A** = Area of the monitored zone in square metres

Fig. 16. Measured crowd density in the demonstrated prototype test.

9.4 Alert Generation Performance

The alert-generation module functioned correctly when the configured critical condition was met. The dashboard showed the crowd-alert state the buzzer activated as expected and the critical alert condition was recorded. The real-time response-time evaluation measured the path across three different configurations. As shown in Figure 16 the average response time was 1.8 seconds for the camera- setup, 1.2 seconds for the laser-sensor-only setup and 0.6 seconds for the combined camera–laser sensor-fusion setup.

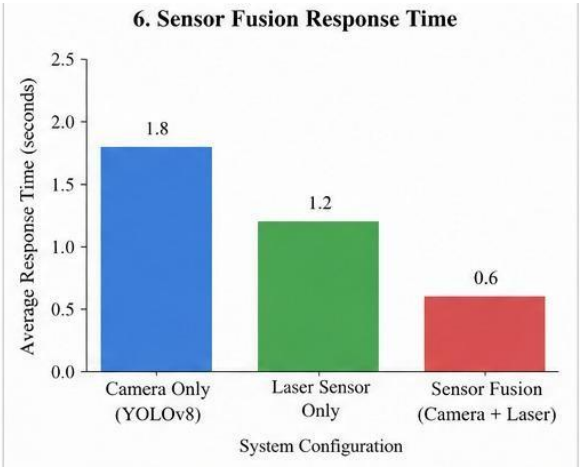


Fig. 16. Average response time, for camera-only laser-sensor and sensor-fusion configurations.

The measured response-time results confirm that the complete sensing-and-decision path worked properly during the evaluation runs. Remote messaging is still an extension of the prototype and would require separate testing to assess latency, availability and recovery from communication failures.

TABLE XI. PROTOTYPE FUNCTIONAL TEST RESULTS

Metric	Observed Result	Scope
Detection success	100% (1/1)	Functional single-person demonstration
People detected	1	Visible person in functional test
Crowd density	0.010 persons/m²	100 m² functional test area
Laser events	4	Beam interruptions
Density rate	0.574 per min	Dashboard observation
Processing speed	18–25 FPS	Observed run range

Metric	Observed Result	Scope
Alert detection	Successful	Functional
Buzzer response	Successful	Functional
Critical alert	Successful	Functional
Tracking	ByteTrack enabled	Functional
Sensor fusion	Camera + Laser	Integrated

9.5 Comparative Analysis

The detection-model evaluation compared YOLOv5, YOLOv8n, YOLOv8s, and YOLOv8m using person-detection and overall mAP@0.5 outputs. Fig. 17 Displays the recorded results for the four model configurations used in the project evaluation. The comparison documents measured behaviour under the project test setup, not a benchmark ranking.

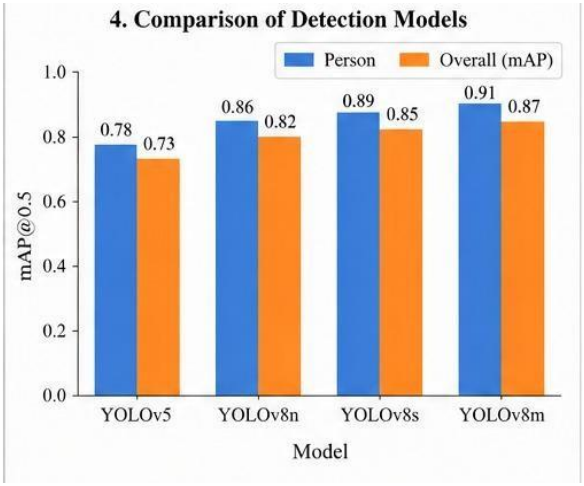


Fig. 17. Comparison of the YOLO model configurations using mAP@0.5 outputs.

A functional comparison was also made across monitoring camera-based AI monitoring, IR/laser-only monitoring and the proposed CrowdGuard AI configuration. CCTV-only systems give information but normally rely on an operator. Camera AI automates detection and tracking. May not provide independent verification of sensor. IR/laser-only systems provide information on crossing but not the whole spatial scene. With the fusion of detection, tracking, sensor support, zone monitoring and automatic alerting in a single pipeline, CrowdGuard AI delivers a comprehensive solution for managing threats in shared indoor spaces.

TABLE XII. FUNCTIONAL COMPARISON OF MONITORING APPROACHES

Approach	Detection	Tracking	Sensor Support	Zone / Alert
CCTV only	Manual	Manual	No	Limited
Camera AI	Yes	Possible	No	Yes / Possible
IR/Laser only	No visual	No	Yes	Point-based / Yes
CrowdGuard AI	Yes	Yes	Yes	Zone-wise / Yes

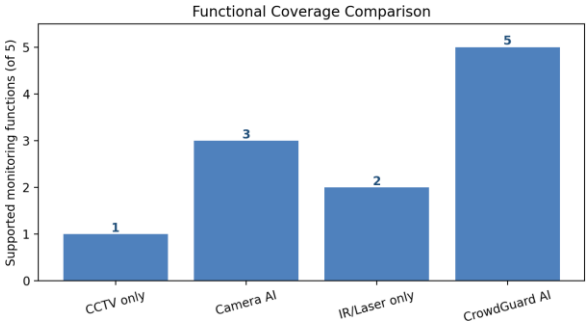


Fig. 18. Functional coverage comparison is based on the capabilities listed in Table XII, which is not a measure of accuracy.

9.6 Discussion of the Demonstrated Result

The real-time evaluation demonstrates that the prototype enables linking the perception, sensing, decision logic, and actuation in a single monitoring loop. The dashboard reveals the state variables immediately, while the evaluation figures offer quantitative results for the detection performance, model comparison and sensor-fusion response, respectively.

It can also be seen from the results that it is important to consider demonstration results separately from larger deployment claims. The 100 m² single person demonstration verifies the end-to-end hardware path and the extra evaluation figures are outputs from the project test runs. The measurements must be repeated in a larger scale field test with different crowd sizes and environmental conditions to confirm the stability of the measurements in the actual test environment.

9.7 Extended Evaluation Plan

The current evaluation is expandable by measuring with a larger number of data and across multiple crowd-density bands. Experimental results should include run number, number of annotated samples, camera configuration, zone area threshold values and environment for each experiment. This protocol may then be used to report mean. Standard deviation for detection metrics such as error, FPS and alert latency. Other field checks include controlled entry and exit sequences, occlusion, changing illumination and multiple monitored zones.

X. ADVANTAGES OF CROWDGUARD AI

Low Cost: The prototype does not require special crowd counting hardware or PC components. Therefore, the design can be replicated for the development and demonstration at locations.

Edge-Oriented Processing: YOLOv8n. Later on, the architecture can be divided into inference nodes and a central dashboard to keep track of all the information.

Fast Detection: The current prototype was tested for the demonstrated run with 1825 FPS. This allows for monitoring to be carried out for the tested scenario

Early Warning: The system generates a report warning before or at the set critical condition based on occupancy, density growth and state.

Easy Installation: The sensor and ESP32 elements can be installed at a desired chokepoint and the camera installed to monitor the corresponding area.

Suitable for Temple Festivals: The architecture is aligned with temple entrances, queue corridors, narrow passages and other high-risk areas. Can be extended to railway stations, malls, stadiums and festivals.

10.1 Operational Value

The operational value of the prototype comes from converting a video stream and sensor events into a compact set of parameters that

can be interpreted quickly. A control-room operator does not need to infer density, from the raw camera frame when the dashboard already provides a count density, laser count, threshold, FPS and status label.This can reduce the amount of visual interpretation required for routine monitoring.

10.2 Scalability

The architecture can be replicated by assigning each chokepoint an identifier, sensor node, camera, zone geometry, and threshold. A central dashboard can then aggregate the zone-level states without requiring every node to send full-resolution video continuously. This provides a potential path from a classroom prototype to a multi-node event-monitoring installation.

XI. LIMITATIONS

There are a number of limitations in the current system which may be taken into account when deploying it. Camera and optical sensor performance can be impacted by outdoor conditions like rain, dust, fog and low light. However, people, banners or temporary structures may obstruct the camera, which will lead to missed detections, especially in crowded locations. If people are using the same devices, this can further decrease detection accuracy. Furthermore, with remote monitoring, delays or communications problems may occur, but there is also a built-in alert system in the local buzzer and LED. The restrictions suggest that more testing is required under various environmental and crowd scenarios prior to widespread deployment.

XII. FUTURE WORK

Future research will be directed towards expanding the application of this system to larger and more difficult deployments in the outdoors. Various cameras and sensor nodes can be installed in various zones and linked to a map-based dashboard for an integrated overview. During night-time and large scale events, thermal cameras and drone based views can be used to enhance monitoring. Other features like direction of movement, track velocity and optical-flow features can also be added to enhance the crowd behavior analysis. A controlled pilot deployment of the system in a public place or temple can further test the system under varying number of people and environmental conditions, focusing on the detection accuracy, reliability of sensor, response time, rate of false alerts, missed alerts, and operator usability.

XIII. CONCLUSION

CrowdGuard AI presents an approach to crowd-density monitoring for temple festivals and similar public gatherings. The system combines YOLOv8n person detection, ByteTrack tracking, zone-counting, IR/laser sensing, threshold checking, trend analysis and local alerts in one IoT-enabled pipeline. The architecture separates computer-vision processing from embedded sensing so that each layer can be developed and tested independently.

The evaluation combines a hardware demonstration with additional real-time test outputs. The functional demonstration detected one out of one person measured 0.010 persons/m² in a 100 m² area recorded four laser events and operated at approximately 18–25 FPS. The evaluation recorded accuracy 0.92 precision 0.89 recall 0.93, F1-score 0.91 and MAE 0.05 for the reported detection/counting outputs. The response-time comparison recorded 1.8 s for camera- processing 1.2 s for the laser-sensor configuration and 0.6 s for sensor fusion. These measurements show the operation of the integrated prototype under the conditions investigated.

The next step is field testing with several people, several crowd sizes, several zones and different environmental conditions. This can be used to verify the repeatability of the reported metrics, sensor-event accuracy, tracking consistency and alert latency during deployment. The target is an extensible crowd-safety solution, to provide event managers with up-to-date quantitative data, yet with a minimal hardware footprint.

APPENDIX

A. HARDWARE AND SOFTWARE CONFIGURATION

This appendix briefly summarizes the hardware and software used in the prototype. The electrical ratings for the components are not given, as they were not documented in the project, and the table is based on the purpose of each component in the system.

Component	Use in CrowdGuard AI	Current Configuration
ESP32	Sensor + alert control	ESP32
ESP32-CAM / Webcam	Video input	Prototype camera
IR / Laser	Crossing detection	Single beam
Buzzer + LED	Local alert	ESP32 controlled
Laptop / PC	AI + dashboard	Windows 10/11
Python / OpenCV	Video processing	3.12 / 5.0.0.93
YOLOv8n / Ultralytics	Person detection	8.4.104
ByteTrack	Multi-object tracking	Enabled
NumPy / PyTorch	Numerical / DL support	2.4.6 / 2.13.0
Arduino IDE / VS Code	Development	2.x / VS Code

B. ALGORITHM PSEUDOCODE

1. Set YOLOv8n, ByteTrack, ESP32 interface, laser counter, zone geometry and threshold.
2. Read the camera frame and the latest ESP32 sensor state.
3. Preprocess the frame and run YOLOv8n person detection.
4. Update ByteTrack and obtain the active track IDs.
5. Count tracked reference points inside the configured monitoring zone.
6. Calculate crowd. The short-window density growth rate.
7. To check the consistency of the sensors, look at the number of camera matches and compare them with the number of laser events recorded.
8. Analyze risk with occupancy, growth and configured thresholds.
9. Update the dashboard. if the situation is critical activate the buzzer/LED. Log the event.
10. Repeat for the frame or time window.

C. ADDITIONAL EXPERIMENTAL RESULTS

The prototype demonstration and the other outputs of the project, acquired during real-time testing, are reported in the present paper. The functional test sets up the integration of the camera, ByteTrack, lasersensor, threshold and local-alert and the extended evaluation delivers detection metrics, crowd-count and density time series, model comparison, confusion-matrix output and responsetime measurements. These values are only applicable for the tested hardware, scenes and evaluation procedure.

Metric	Observed Result	Interpretation
Detection success	100% (1/1)	Single-person case
People detected	1	Visible person
Crowd density	0.010 persons/m²	100 m² area
Laser events	4	Beam interruptions
Processing speed	18–25 FPS	Observed run
Test threshold	1 person	Configured
ByteTrack	Enabled	Tracking active
Sensor fusion	Successful	Camera + laser
Alerts / buzzer	Successful	Critical response

Repeated clips under medium and high crowd conditions, with multiple environmental conditions and verified ground truth should be used for further validation. The following study should present mean and deviation for precision, recall, F1-score, mAP, counting error, tracking consistency and alert latency on a test set aside for this purpose. The existing measurements should thus be considered as measured project results with the test protocol, and not as universal deployment guarantees.

REFERENCES

[1] Z. Chen, X. Xie, T. Qiu, and L. Yao, "Dense-Stream YOLOv8n: A Lightweight Framework for Real-Time Crowd Monitoring in Smart Libraries," Scientific Reports, vol. 15, Art. no. 11618, 2025, doi: 10.1038/s41598-025-94659-x.

[2] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in Computer Vision – ECCV 2022, pp. 1–21, 2022, doi: 10.1007/978-3-031-20047-2_1.

[3] Y. Li, X. Zhang, and D. Chen, "CSRNet: Dilated Convolutional Neural Networks for Understanding the Highly Congested Scenes," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2018, pp. 1091–1100, doi: 10.1109/CVPR.2018.00120.

[4] B. Pardamean, F. Abid, T. W. Cenggoro, G. N. Elwirehardja, and H. H. Muljo, "Counting people inside a region-of-interest in CCTV footage with deep learning," PeerJ Computer Science, vol. 8, e1067, 2022, doi: 10.7717/peerj-cs.1067.

[5] H. H. Nguyen, V. T. Bui, T. N. Ta, H. M. Pham, N. C. Nguyen, and D. M. Nguyen, "YOLO Based Real-Time Human Detection for Smart Video Surveillance at the Edge," in Proc. IEEE Int. Conf. Communications and Electronics (ICCE), 2021, pp. 439–444, doi: 10.1109/ICCE48956.2021.9352144.

[6] J. Reddy et al., "Crowd Control and Monitoring using Deep Learning," in Proc. 2024 Second Int. Conf. Networks, Multimedia and Information Technology (NMITCON), 2024, doi: 10.1109/NMITCON62075.2024.10698968.

[7] S. Wang, Z. Pu, Q. Li, and Y. Wang, "Estimating crowd density with edge intelligence based on lightweight convolutional neural networks," Expert Systems with Applications, vol. 206, Art. no. 117823, 2022, doi: 10.1016/j.eswa.2022.117823.

[8] A. C. Cob-Parro et al., "Real-time stampede detection using optical flow and machine learning techniques," IEEE Access, vol. 9, 2021.

[9] L. Boominathan, S. S. S. Kruthiventi, and R. Venkatesh Babu, "CrowdNet: A Deep Convolutional Network for Dense Crowd Counting," in Proc. ACM Int. Conf. on Multimedia, 2016.

[10] Y. Zhang, D. Zhou, S. Chen, S. Gao, and Y. Ma, "Single-Image Crowd Counting via Multi-Column Convolutional Neural Network," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 589–597, doi: 10.1109/CVPR.2016.70.

[11] H. Idrees, I. Saleemi, C. Seibert, and M. Shah, "Multi-Source Multi-Scale Counting in Extremely Dense Crowd Images," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2013, pp. 2547–2554, doi: 10.1109/CVPR.2013.329.

[12] V. A. Sindagi and V. M. Patel, "A survey of recent advances in CNN-based single image crowd counting and density estimation," Pattern Recognition Letters, vol. 107, pp. 3–16, 2018, doi: 10.1016/j.patrec.2017.07.007.

[13] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.

[14] J. Terven, D.-M. Cordova-Esparza, and J.-A. Romero-Gonzalez, "A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS," Machine Learning and Knowledge Extraction, vol. 5, no. 4, pp. 1680–1716, 2023, doi: 10.3390/make5040083.

[15] A. Milan et al., "MOTChallenge: A Benchmark for Single-Camera Multiple Target Tracking," International Journal of Computer Vision, vol. 129, pp. 834–849, 2021, doi: 10.1007/s11263-020-01393-0.

[16] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," in Proc. IEEE Int. Conf. on Image Processing (ICIP), 2017, pp. 3645–3649, doi: 10.1109/ICIP.2017.8296962.

[17] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," in Advances in Neural Information Processing Systems, vol. 28, 2015, pp. 91–99.

[18] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," in Computer Vision – ECCV 2014, 2014, pp. 740–755, doi: 10.1007/978-3-319-10602-1_48.

[19] D. R. Arun, C. C. Columbus, A. Bhuvanesh, and A. S. Sumithra, "Smart Crowd Monitoring System Using IoT-Based YOLO-GHOST," Rev. Roum. Sci. Techn.– Électrotechn. et Énerg., vol. 69, no. 3, pp. 341–346, 2024, doi: 10.59277/RRST-EE.2024.69.3.15.

[20] A. M. Al-Shaery et al., "Crowd management challenges: A tackling approach using multi-sensor fusion," IEEE Access, 2019.

[21] F. Aldosari, S. Alamri, R. Alharbi, and G. Alghamdi, "IoT based crowd management tool," in Proc. SAI Int. Systems Conf., 2022, pp. 533–539.

[22] N. Thakur, P. Nagrath, R. Jain, D. Saini, N. Sharma, and D. J. Hemanth, "Autonomous pedestrian detection for crowd surveillance using deep learning framework," Soft Computing, vol. 27, no. 14, pp. 9383–9399, 2023.

[23] M. Ş. Gündüz and G. Işık, "A new YOLO-based method for real-time crowd detection from video and performance analysis of YOLO models," Journal of Real-Time Image Processing, vol. 20, 2023.

[24] L. Boominathan, S. S. S. Kruthiventi, and R. Venkatesh Babu, "CrowdNet: A Deep Convolutional Network for Dense Crowd Counting," in Proc. 2016 ACM Multimedia Conference, 2016, pp. 640–644.

[25] C. Wang, H. Zhang, L. Yang, S. Liu, and X. Cao, "Deep People Counting in Extremely Dense Crowds," in Proc. 23rd ACM Int. Conf. on Multimedia, 2015, pp. 1299–1302.

[26] C. Zhang, H. Li, X. Wang, and X. Yang, "Cross-Scene Crowd Counting via Deep Convolutional Neural Networks," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2015, pp. 833–841.

[27] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770–778.

[28] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in Advances in Neural Information Processing Systems, vol. 32, 2019, pp. 8026–8037.

[29] S. Shao, Z. Zhao, B. Li, T. Xiao, G. Yu, X. Zhang, and J. Sun, “CrowdHuman: A Benchmark for Detecting Human in a Crowd,” arXiv preprint arXiv:1805.00123, 2018.

[30] P. Thanasutives, K.-I. Fukui, M. Numao, and C. Kijsirikul, “Encoder-Decoder Based Convolutional Neural Networks with Multi-Scale-Aware Modules for Crowd Counting,” arXiv preprint arXiv:2003.05586, 2020.